Claude Code pour Coline

Tout ce que Julien te transmet, et comment l'utiliser sans vider ton abonnement.

Kit préparé le 22 octobre 2026 · Abonnement Claude Pro · Mac + VS Code

---

Ce guide accompagne le kit installé sur ton Mac. Il explique ce que chaque pièce fait, pourquoi elle existe, et la routine à suivre au quotidien. Lis-le une fois en entier (20 minutes), puis garde la dernière page sous la main : c'est l'aide-mémoire.

> **Le principe en une phrase.** Tu diriges, Claude exécute. Tu réfléchis avec lui dans une session courte, il écrit un plan en petites étapes, puis tu lances chaque étape dans une conversation neuve. C'est ce qui permet de faire beaucoup avec un petit compteur.

Sommaire

1. Ce qui a été installé sur ton Mac
2. La barre de statut : lire ta consommation en direct
3. Pourquoi les tokens filent, et les trois réflexes
4. Les modèles : lequel pour quoi
5. La méthode : brainstorm, plan, blocs
6. Première séance pas à pas : refaire une page de ta boutique
7. Les vidéos avec HyperFrames
8. Sécurité : ce que Claude ne fera jamais sans toi
9. Sauvegarder avec GitHub
10. Dépannage
11. Aide-mémoire

1. Ce qui a été installé sur ton Mac

La commande que tu as collée a récupéré le dépôt GitHub de Julien dans `~/KIT-CLAUDE-CODE`, lancé l'installation, puis ouvert ce guide. Voici ce qui a été copié dans le dossier caché `~/.claude` (le dossier de réglages de Claude Code) :

Fichier	Rôle
`~/.claude/CLAUDE.md`	Tes règles permanentes. Claude les lit au début de chaque conversation, dans tous tes projets. C'est là que tu dis qui tu es, ce que tu fais, et comment tu veux qu'il travaille.
`~/.claude/settings.json`	Les réglages : Sonnet comme modèle par défaut, la barre de statut, le mode de permissions.
`~/.claude/scripts/statusline.sh`	Le script de la barre de statut (section 2).
`~/.claude/scripts/usage.sh`	Affiche ta consommation dans le Terminal avec un conseil, quand Claude Code n'est pas ouvert.
`~/.claude/commands/brainstorm.md`, `plan.md`, `bloc.md`	Les trois commandes de la méthode (section 5). Tu les tapes avec un `/` devant.

Rien de ce qui existait avant n'a été effacé : si tu avais déjà un `CLAUDE.md` ou un `settings.json`, une copie datée a été gardée à côté (suffixe `.sauvegarde-…`).

Le dossier `~/KIT-CLAUDE-CODE` contient aussi `video/`, un modèle de studio vidéo à copier dans tes projets (section 7), et le fichier `LISEZMOI.md`, version courte de ce guide.

À faire maintenant : personnaliser tes règles

Ouvre `~/.claude/CLAUDE.md` dans VS Code (Fichier, Ouvrir… ; tape Cmd+Maj+. dans la fenêtre d'ouverture pour voir les dossiers cachés). Remplace les crochets :

- `[TA MARQUE]` : le nom de ta marque.
- `[PLATEFORME : Shopify, WooCommerce, Wix...]` : l'outil de ta boutique.
- `[URL]` : l'adresse de ton site.

Plus Claude sait qui tu es, moins tu dois le réexpliquer, et moins tu dépenses.

2. La barre de statut : lire ta consommation en direct

Sous la zone où tu écris à Claude, une ligne s'affiche en permanence :

```
🤖 Sonnet 5.5 | 🧠 ctx: 12% | 🕐 5h: 34% · reset 23/10 02h00 | 📅 Sem: 41% · reset 29/10 20h00
```

Élément	Ce que c'est	Quoi faire
🤖 Sonnet 5.5	Le modèle qui répond en ce moment.	Vérifie qu'il correspond à ce que tu fais (section 4).
🧠 ctx	Le remplissage de la conversation en cours. Claude relit toute la conversation à chaque message : plus elle est pleine, plus chaque réponse coûte.	Sous 40 % : tranquille. 40 à 60 % : un « ! » apparaît, termine ce que tu fais. Au-dessus de 60 % : ⚠️, tape `/clear`.
🕐 5h	Ta fenêtre de 5 heures. Elle se remplit avec chaque réponse (chat Claude inclus) et se vide d'un coup à l'heure indiquée.	À 80 %, un ⚠️ apparaît : finis proprement et attends la remise à zéro.
📅 Sem	Ta limite hebdomadaire. C'est le vrai plafond : quand elle est pleine, plus rien jusqu'à la date indiquée.	Répartis : si tu es à 50 % le mardi, ralentis. Garde toujours 15 % pour un imprévu.

Les chiffres 5 h et Sem viennent de Claude Code lui-même et sont les mêmes que sur claude.ai. Ils apparaissent après ton premier message dans une session. La commande `/usage` affiche les mêmes informations en grand.

3. Pourquoi les tokens filent, et les trois réflexes

Un « token » est un morceau de mot. Tout ce que tu écris, tout ce que Claude lit (fichiers, résultats de commandes) et tout ce qu'il répond consomme des tokens. Deux choses coûtent cher sans qu'on s'en rende compte :

1. **Les longues conversations.** Au vingtième message, Claude relit les dix-neuf précédents avant de répondre. Une conversation de deux heures peut coûter dix fois plus qu'une conversation de dix minutes avec le même résultat.
2. **Le modèle puissant pour une tâche simple.** Opus consomme nettement plus que Sonnet (environ le double d'après l'expérience de Julien) pour corriger une faute d'orthographe aussi bien que Sonnet.

> **Les trois réflexes.**
> 1. **Un sujet, une session.** Nouveau sujet ? `/clear`. Tu ne perds rien : la mémoire du projet est dans les fichiers BRIEF.md et PLAN.md, pas dans la conversation.
> 2. **Le bon modèle.** Sonnet pour produire (ton défaut), Opus pour réfléchir (brainstorm, plan), Haiku pour ranger.
> 3. **Écrire la mémoire dans des fichiers.** Quand une décision est prise, elle va dans BRIEF.md ou PLAN.md. La prochaine session la lit en quelques secondes.

Autres habitudes qui comptent : interrompre Claude avec Échap dès qu'il part dans une mauvaise direction (chaque seconde de travail inutile coûte), ne pas lui demander de « tout relire » ou de « tout

vérifier » sans raison, et ne pas coller des pages entières de texte quand un fichier dans le dossier suffit.

4. Les modèles : lequel pour quoi

Modèle	Commande	Quand	Coût
Sonnet 5.5	`/model sonnet`	Produire : pages, textes de boutique, fiches produit, visuels, vidéos, corrections, réglages. Ton modèle par défaut.	Référence
Opus 5.5	`/model opus`	Réfléchir : brainstorm, plan, décision importante, problème que Sonnet n'arrive pas à résoudre. Sessions courtes.	Environ 2×
Haiku 5.5	`/model haiku`	Mécanique : renommer des fichiers, ranger, mettre en forme, résumer un document. Jamais de texte pour tes clients.	Très faible

Le modèle Fable qu'utilise Julien n'est pas inclus dans l'abonnement Pro. Tu n'en as pas besoin : Opus est très largement suffisant pour réfléchir à un site, une boutique ou une vidéo.

L'effort. En plus du modèle, tu peux régler la profondeur de réflexion : `/effort medium` pour du contenu et des tâches simples (moins de tokens), `/effort high` pour un bloc difficile. Le kit met `high` par défaut ; passe en `medium` quand tu écris du texte ou que tu fais des petites retouches.

5. La méthode : brainstorm, plan, blocs

C'est la façon de travailler de Julien, adaptée à ton abonnement. Trois commandes, trois sessions différentes, toujours dans le dossier du projet (dans VS Code : Fichier, Ouvrir le dossier…).

Étape 1 : `/brainstorm` (Opus)

Tu donnes une idée en une phrase. Claude ne te bombarde pas de questions : il propose d'abord une direction complète (pour qui, quelle promesse, quelles pages ou vidéos, quelles idées signature marquées [IDÉE]), puis il pose ses questions en une seule fois, par choix multiples. Il écrit au fil de l'eau un fichier `BRIEF.md` à la racine du dossier. Quand le brief est prêt, il te le dit et te donne la marche à suivre.

```
/model opus
/brainstorm "refaire la page d'accueil pour mettre en avant la collection d'hiver"
```

Étape 2 : `/plan` (Opus, session neuve)

Après `/clear`, Claude lit seulement BRIEF.md et écrit `PLAN.md` : une suite de blocs de 20 minutes à 1 heure de travail, chacun avec un résultat que tu peux voir et tester, le modèle à utiliser, ce que tu dois fournir avant, et la marche à suivre exacte. Il propose aussi une répartition (« blocs 0 et 1 aujourd'hui, 2 et 3 demain ») en regardant ta barre de statut.

```
/clear
/model opus
/plan
```

Étape 3 : `/bloc N` (Sonnet en général, session neuve à chaque bloc)

Après `/clear`, Claude lit PLAN.md, exécute le bloc demandé, teste le résultat, met à jour le plan (statut, reste à finir, journal), et te rend compte : ce qu'il a fait, ce que tu dois tester, le prochain bloc. Puis il s'arrête et attend ton « ok ». Il n'enchaîne jamais tout seul.

```
/clear
/model sonnet
/bloc 1
```

> **Pourquoi ça économise autant.** Chaque session repart de zéro et ne lit qu'un fichier court. La conversation reste petite, donc chaque message reste bon marché. Et si tu fermes VS Code au milieu, rien n'est perdu : le plan dit exactement où tu en es.

Un bloc n'a pas besoin d'être parfait pour être validé. Les petits détails vont dans la rubrique « Reste à finir » du plan et se reprennent plus tard, groupés, dans un bloc Sonnet ou Haiku.

Quand Claude bloque. Les commandes lui imposent de s'arrêter après trois essais ratés sur le même problème et de t'expliquer deux ou trois pistes différentes avec leurs risques. C'est toi qui choisis. Ça évite les boucles qui vident la fenêtre de 5 heures pour rien.

6. Première séance pas à pas : refaire une page de ta boutique

1. Crée un dossier pour le projet, par exemple `~/Documents/PROJETS/BOUTIQUE`. Jamais dans iCloud Drive, OneDrive ou Google Drive (les fichiers techniques y créent des conflits). Mets-y ce qui existe : captures d'écran de la page actuelle, textes, photos, logo.
2. Dans VS Code : Fichier, Ouvrir le dossier…, choisis ce dossier. Ouvre le panneau Claude Code.
3. Regarde la barre : si « 5h » est au-dessus de 60 %, attends la remise à zéro avant de commencer un brainstorm.
4. Tape `/model opus` puis `/brainstorm "refaire la page d'accueil de ma boutique pour la collection d'hiver"`. Réponds aux questions groupées. Compte 10 à 20 minutes.
5. Quand Claude dit « prêt pour /plan » : `/clear`, `/model opus`, `/plan`. Lis le plan : la liste « Ce que je dois fournir » te dit ce qu'il attend de toi (accès à la boutique, photos, textes). Fournis-le dans le dossier.
6. `/clear`, `/model sonnet`, `/bloc 0`. Le bloc 0 est toujours la mise en place et la sauvegarde de l'existant. Valide.
7. Pour chaque bloc suivant : `/clear`, le modèle indiqué dans le plan, `/bloc N`. Teste ce que Claude te demande de tester. Dis « ok » ou ce qui ne va pas.
8. Quand une mise en ligne réelle sur ta boutique est en jeu, Claude te montre d'abord et attend ton accord explicite. Si ta plateforme n'est pas pilotable depuis un fichier (Shopify, Wix…), il te prépare les textes et visuels prêts à coller, avec l'ordre exact des manipulations.

> **Un conseil de Julien.** Commence par un petit projet (une page, cinq fiches produit) pour apprendre la méthode. Pas la refonte complète du site le premier jour.

7. Les vidéos avec HyperFrames

Julien fabrique ses vidéos de motion design (textes animés, transitions, musique, carte de fin) avec Claude Code et HyperFrames, un outil gratuit : la vidéo est décrite comme une page web animée, puis rendue en MP4. Pas de montage à la main. Le dossier `~/KIT-CLAUDE-CODE/video` reproduit sa méthode avec une commande `/video` adaptée à ta marque.

Installation (une fois)

1. Installe Node.js version LTS depuis `https://nodejs.org` si ce n'est pas déjà fait.
2. Copie le dossier `video` dans tes projets, par exemple `~/Documents/PROJETS/VIDEOS`.
3. Ouvre ce dossier dans VS Code, lance Claude Code et colle :

   ```
   Installe HyperFrames dans ce dossier avec `npx hyperframes skills` (le jeu
   complet de skills), puis vérifie avec `npx hyperframes skills check`. Ne crée
   aucune vidéo pour l'instant.
   ```

4. Remplis `kit/marque/frame.md` (couleurs, polices, ton, carte de fin) et `kit/produit/PRODUIT.md` (ta marque, tes produits, tes messages exacts, les seuls chiffres autorisés). Mets ton logo dans `kit/logos/` et des photos ou captures réelles dans `kit/captures/`. Claude n'invente ni chiffre ni visuel : tout vient de ce kit.

Produire une vidéo

```
/clear
/model sonnet
/video "montrer le déballage de la nouvelle collection, ambiance douce, 20
secondes, pour Instagram"
```

Claude s'arrête deux fois : il te montre le script (60 mots au plus, une accroche, une idée, une preuve réelle, un appel) et attend ton « ok » ; puis il te montre une planche de 4 ou 5 images clés et attend ton « ok ». Ensuite il construit, rend `videos/<nom>/renders/video.mp4`, et fait un contrôle qualité (lisibilité, zone sûre, carte de fin). Pour t'inspirer d'une vidéo vue sur Instagram ou TikTok, décris dans le prompt ce que tu aimes (rythme, taille des textes, ambiance) : il s'en inspire sans copier.

> **Attention à la consommation.** Une vidéo est le travail le plus gourmand du kit : script, direction artistique, composition, contrôle. Compte qu'une vidéo peut occuper une grande partie d'une fenêtre de 5 heures. Règles : une vidéo par session, jamais deux ; lancer seulement quand « 5h » est sous 30 % et « Sem » sous 60 % ; une seule version par vidéo puis des retouches ciblées ; Sonnet par défaut, Opus seulement si le résultat déçoit vraiment.

Si Claude dit qu'un besoin n'est couvert par aucun skill (musique, image générée, effet particulier), il s'arrête et te propose une solution : c'est voulu, il ne doit pas bricoler. Toute génération payante (image, musique) demande ton accord explicite avant.

8. Sécurité : ce que Claude ne fera jamais sans toi

Les règles permanentes du kit lui interdisent, dans tous tes projets :

- de supprimer un fichier avec `rm` (il passe par la corbeille ou te demande) ;
- de modifier ta boutique en ligne réelle, publier sur un réseau, envoyer un email, payer ou lancer une génération payante sans te l'avoir montré d'abord et sans ton « ok » explicite ;
- d'afficher ou copier une clé API, un mot de passe, un fichier `.env` ;
- de livrer un bouton qui ne fait rien, un texte « à compléter » ou une tâche à moitié finie en disant que c'est terminé ;
- d'inventer un chiffre, un prix, un avis client, une date.

Le mode de permissions. Le kit règle Claude Code en mode `acceptEdits` : il peut créer et modifier les fichiers du projet sans te demander, mais il te demande confirmation avant toute commande système. Quand une demande apparaît pour une commande sûre que tu reverras souvent, réponds « Yes, and don't ask again » : la permission est mémorisée. Quand tu seras à l'aise, Julien pourra te montrer le mode « bypass » qui ne demande plus rien, avec un garde-fou adapté.

Les sauvegardes. Avant tout changement risqué sur un fichier existant, Claude fait une copie datée (ou un commit Git si le dossier en a un). Le bloc 0 de chaque plan sauvegarde l'existant.

9. Sauvegarder avec GitHub

GitHub est un coffre en ligne pour tes dossiers de projet, avec l'historique de chaque version. Tu as un compte (`Colinechartier`). Pour y mettre un projet, dis à Claude dans le dossier du projet :

```
Initialise Git dans ce dossier, crée un dépôt privé sur mon compte GitHub
Colinechartier et pousse le contenu. Explique-moi chaque étape en une ligne.
```

La première fois, il te fera te connecter avec `gh auth login` (un code à recopier dans le navigateur). Ensuite, à la fin d'un bloc, « commit et push » suffit : le plan et le travail sont à l'abri. Les photos et vidéos lourdes n'ont pas leur place sur GitHub : garde-les dans le dossier, Claude les exclura.

10. Dépannage

Symptôme	Cause probable	Solution
La barre n'apparaît pas	Claude Code n'a pas rechargé les réglages	Ferme et relance VS Code. Vérifie `~/.claude/settings.json` contient bien `statusLine`.
La barre affiche des tirets partout	`jq` absent, ou pas connectée	Dans le Terminal : `jq --version`. S'il manque : `brew install jq`. Puis `/login` dans Claude Code. Les chiffres 5 h et Sem n'apparaissent qu'après ton premier message.
« Not logged in »	Session expirée	Tape `/login` et reconnecte-toi avec ton compte Pro.
`/brainstorm` inconnu	Les commandes ne sont pas chargées	Vérifie que `~/.claude/commands/brainstorm.md` existe. Relance VS Code.
Claude demande une autorisation à chaque action	Mode `acceptEdits`, normal pour les commandes système	« Yes, and don't ask again » pour les commandes sûres et répétitives.
Claude tourne en boucle sur un bug	Il n'a pas appliqué la règle des trois essais	Échap, puis écris : « Stop. Trois essais. Donne-moi deux pistes différentes et laisse-moi choisir. »
« ctx » monte très vite	Claude lit de gros fichiers ou de longs résultats	Demande-lui de ne lire que ce dont le bloc a besoin. Si tu dépasses 60 % : il met le journal du plan à jour, puis `/clear` et `/bloc N` pour reprendre.
La fenêtre 5 h est pleine	Trop de travail d'un coup	L'heure de remise à zéro est dans la barre. Profites-en pour tester ce qui a été fait et préparer le prochain bloc.
Rien ne marche		Envoie à Julien une capture de la barre et le message d'erreur exact.

11. Aide-mémoire

La routine

1. Ouvrir le dossier du projet dans VS Code.
2. Regarder la barre : « 5h » et « Sem » laissent-ils de la marge ?
3. Réfléchir : `/model opus`, `/brainstorm "idée"`.
4. Découper : `/clear`, `/model opus`, `/plan`.
5. Produire : `/clear`, `/model sonnet`, `/bloc N`. Tester. Valider. Recommencer au bloc suivant.
6. Entre deux sujets : toujours `/clear`.

Les commandes

Commande	Effet
`/clear`	Nouvelle conversation vide. Le réflexe numéro 1.
`/usage`	Consommation 5 h et semaine, en grand.
`/model sonnet` · `/model opus` · `/model haiku`	Changer de modèle (production · réflexion · mécanique).
`/effort medium` · `/effort high`	Moins ou plus de réflexion. Medium pour le contenu simple.
`/context`	Voir ce qui remplit la conversation.
`/compact`	Résumer la conversation. Préférer `/clear` quand le plan est à jour.
`/login`	Se reconnecter.
Échap	Interrompre Claude immédiatement.
`/brainstorm "idée"`	Direction complète + questions groupées + BRIEF.md (Opus).
`/plan`	BRIEF.md devient PLAN.md en blocs courts (Opus, session neuve).
`/bloc N`	Exécute un bloc, teste, met le plan à jour, attend ton ok (Sonnet, session neuve).
`/video "idée"`	Dans le dossier vidéo : une vidéo en deux arrêts (script, planche), puis MP4.

Les seuils

Indicateur	Vert	Orange	Rouge
🧠 ctx	sous 40 %	40 à 60 % : terminer l'étape	plus de 60 % : `/clear`
🕐 5h	sous 60 %	60 à 80 % : pas de nouveau gros chantier	plus de 80 % : finir, puis attendre le reset
📅 Sem	sous 60 %	60 à 80 % : essentiel seulement	plus de 80 % : garder pour l'urgent
Vidéo	Lancer seulement si 5h sous 30 % et Sem sous 60 %. Une vidéo par session.		

Ce guide est dans `~/KIT-CLAUDE-CODE/guide/GUIDE-COLINE.pdf`. Pour le rouvrir : double-clic, ou dans le Terminal `open ~/KIT-CLAUDE-CODE/guide/GUIDE-COLINE.pdf`. Pour mettre le kit à jour quand Julien le modifie : `cd ~/KIT-CLAUDE-CODE && git pull && bash install.sh`.